

并行计算

■ 课程信息

- ▶ 任课教师：计算机与软件学院 陆克中
- ▶ Email: kzLu@qq.com
- ▶ 微信面对面建群：

■ 成绩评定

- ▶ 30%实验报告
- ▶ 10%课后作业
- ▶ 60%期末考试（闭卷）

■ 教材

- ▶ 彼得·S.帕切科, 马修·马伦塞克. 并程序序设计导论（第2版）. 北京: 机械工业出版社, 2024.

■ 参考书

- ▶ 陈国良. 并行计算—结构.算法.编程（第3版）. 北京: 高等教育出版社, 2011.



第1章 为什么需要并行计算?

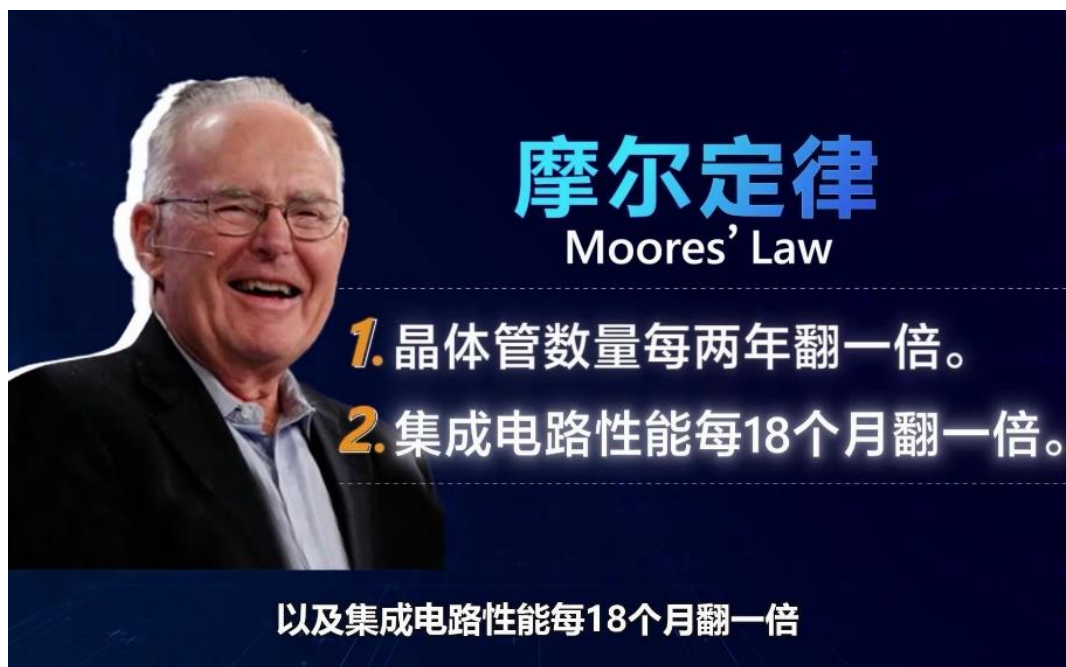
深圳大学计算机与软件学院 陆克中

路线图

- 为什么需要不断提高性能?
- 为什么需要建立并行系统?
- 为什么需要编写并行程序?
- 如何编写并行程序?
- 我们将做什么?
- 并发、并行和分布式。

变化的时代

- 从1986年到2003年，微处理器的速度像火箭一样飞速提升，性能平均每年增加50%以上。
- 然而，自2003年以来，单核处理器的性能提升已经逐步放缓，在2015年至2017年期间，每年的提升幅度不到4%。



并行化方法

- 制造商不再试图继续开发越来越快的单核处理器，而是开始尝试将多个完整的处理器核心放在一个集成电路上。



英特尔® 奔腾® 4 处理器 2.53 GHz, 512K高速缓存, 130nm 光刻, 2000年



Intel® Core™ i9 processor 14900K 36M Cache, up to 6.00 GHz, 24 核, 7nm光刻, 2023年

现在就看程序员的了

- 如果程序员意识不到多核处理器的存在，单纯增加处理器数并没多大用处.....
-或者不知道如何使用它们。
- 在大多数情况下，串行程序不能从这种方法中受益。



为什么需要不断提高性能

- 随着计算能力的提高，我们可以认真考虑和解决的问题的数量也在增加。
- 由于过去的进步，我们从未想象过的问题已经得到解决，例如人类基因组解码。
- 更复杂的问题仍有待解决。

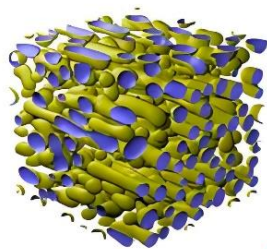
数千年来，科学研究跨越了**实验观测**、**理论推演**、**计算仿真**几个阶段，正进入**数据密集型的“第四范式”**。



几千年前，
实验观测

$$\left(\frac{\dot{a}}{a}\right)^2 = \frac{4\pi G\rho}{3} - K\frac{c^2}{a^2}$$

几百年前，
理论推演



几十年前，
计算仿真



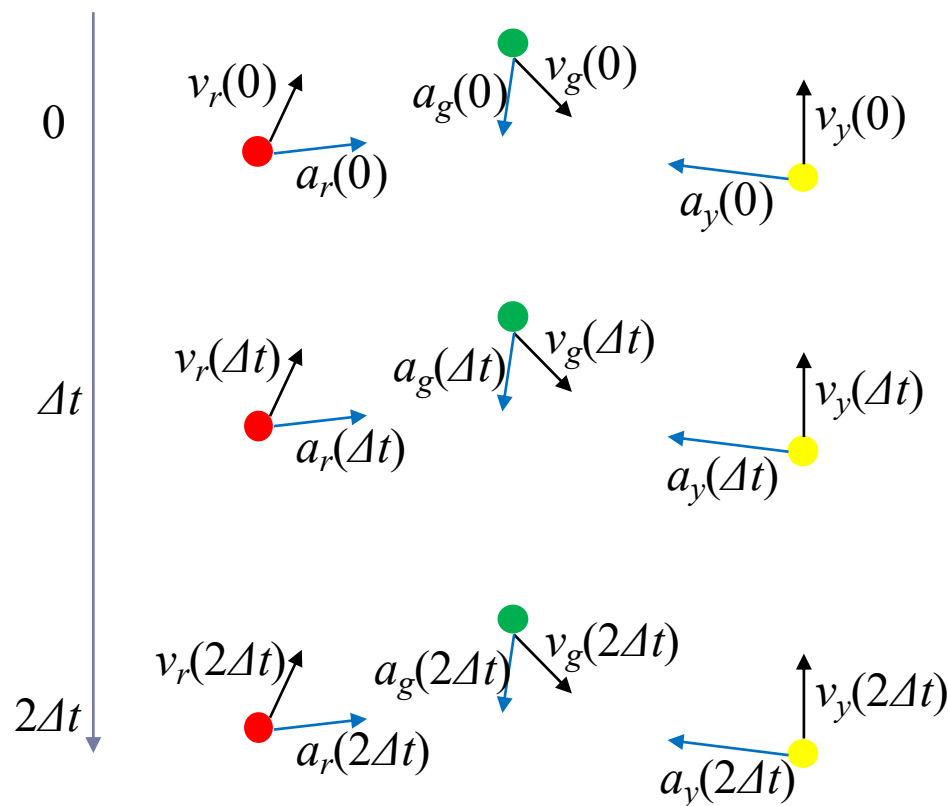
正进入数
据密集型



The
**FOURTH
PARADIGM**
DATA-INTENSIVE SCIENTIFIC DISCOVERY

N体问题

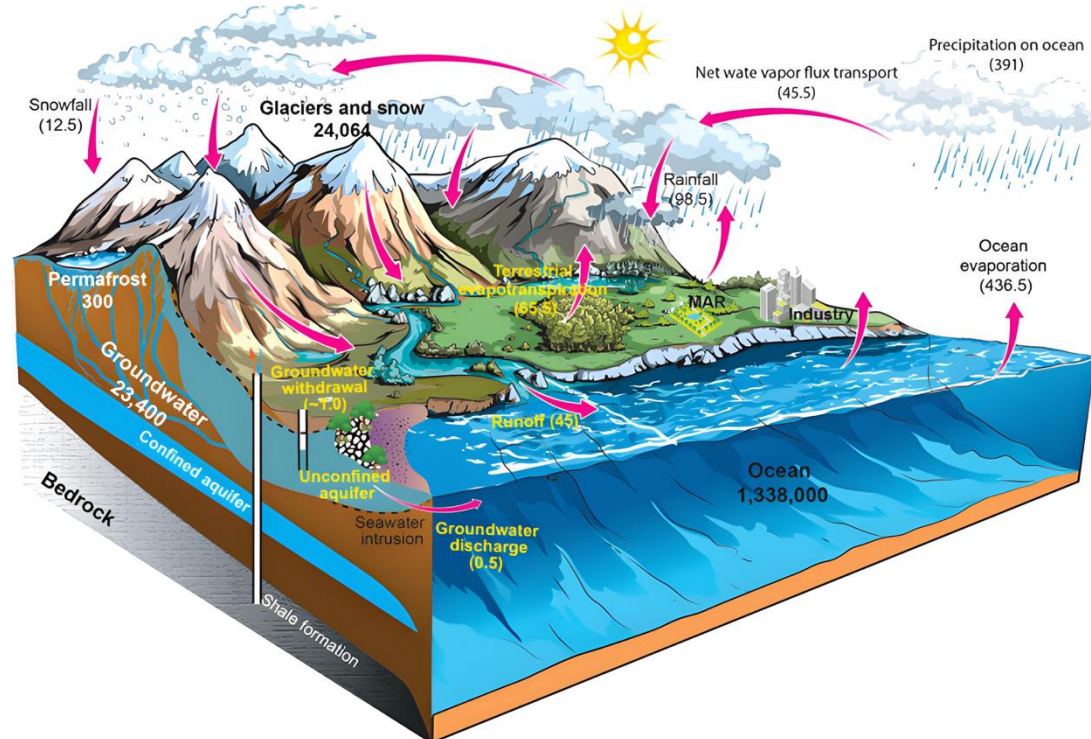
- N个质点在牛顿万有引力作用下的运动规律，已知初始位置与速度，求解任意时刻的位置与速度。
- 两体问题 ($N=2$)
 - ▶ 完全可解
- 三体及以上 ($N\geq 3$)
 - ▶ 无通用解析解
 - ▶ 基于牛顿经典力学原理
 - ▶ 采用迭代法进行数值模拟



气候建模

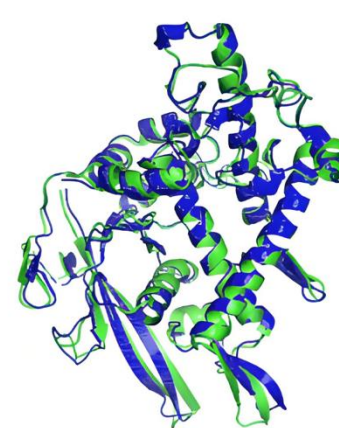
■ NeuralGCM

- ▶ 谷歌研究推出的全球大气混合环流模型
- ▶ 将传统物理方程与深度学习结合
- ▶ 可以在30秒内生成22.8天的大气模拟

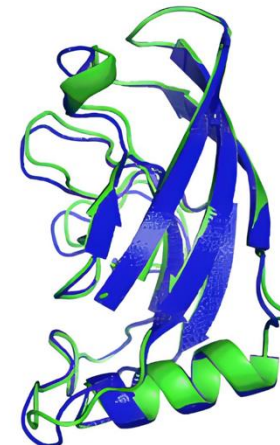


蛋白质折叠

- 蛋白质折叠问题一直是生物學界的重大挑战。
 - ▶ 利用X射线晶体学或冷冻电镜等实验技术来破译蛋白质的三维结构。
 - ▶ 耗时长、成本高。
- AlphaFold 2
 - ▶ DeepMind公司开发。
 - ▶ 开发者Demis Hassabis和John Jumper获得诺贝尔化学奖。



T1037 / 6vr4
90.7 GDT
(RNA polymerase domain)



T1049 / 6y4f
93.3 GDT
(adhesin tip)

● Experimental result
● Computational prediction

药物发现

- 2026年初，英伟达与礼来联手投资10亿美元打造AI制药实验室。
 - ▶ 过去二十年，新药研发成本翻了三倍，平均耗时超过10年，成功率却不足10%。
 - ▶ AI在计算环境中模拟分子行为，指导自动化湿实验室进行实验
 - ▶ 实验结果再反馈给AI模型，实现持续学习与优化



能源研究



数据分析

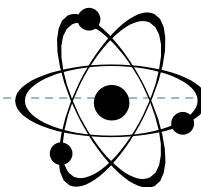


为什么需要建立并行系统

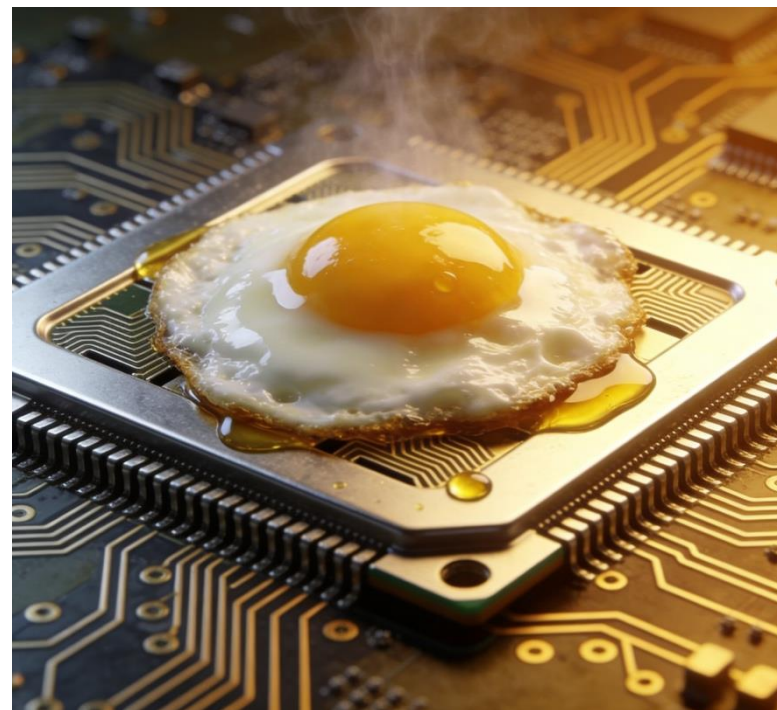
- 单处理器性能的巨大提高很大程度上是由不断增大的晶体管密度推动的。
- 但其中存在着内在缺陷。



小小的物理课



- 更小的晶体管 = 更快的处理器。
- 处理器速度更快 = 功耗增加。
- 功耗增加 = 热量增加。
- 温度升高 = 处理器不可靠。



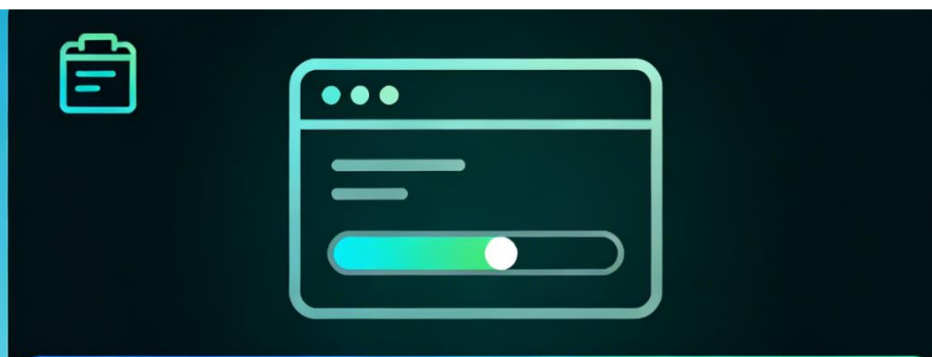
解决方案

- 从单核处理器转向多核处理器。
- Core = 中央处理单元 (CPU)



单核性能瓶颈

功耗限制：频率提升导致功耗指数级增长
散热难题：高功耗带来散热压力
成本增加：提升单核性能需更复杂工艺



多核并行优势

多任务处理：同时运行多个程序不卡顿
效率提升：并行计算缩短处理时间
扩展性强：增加核心数可线性提升性能

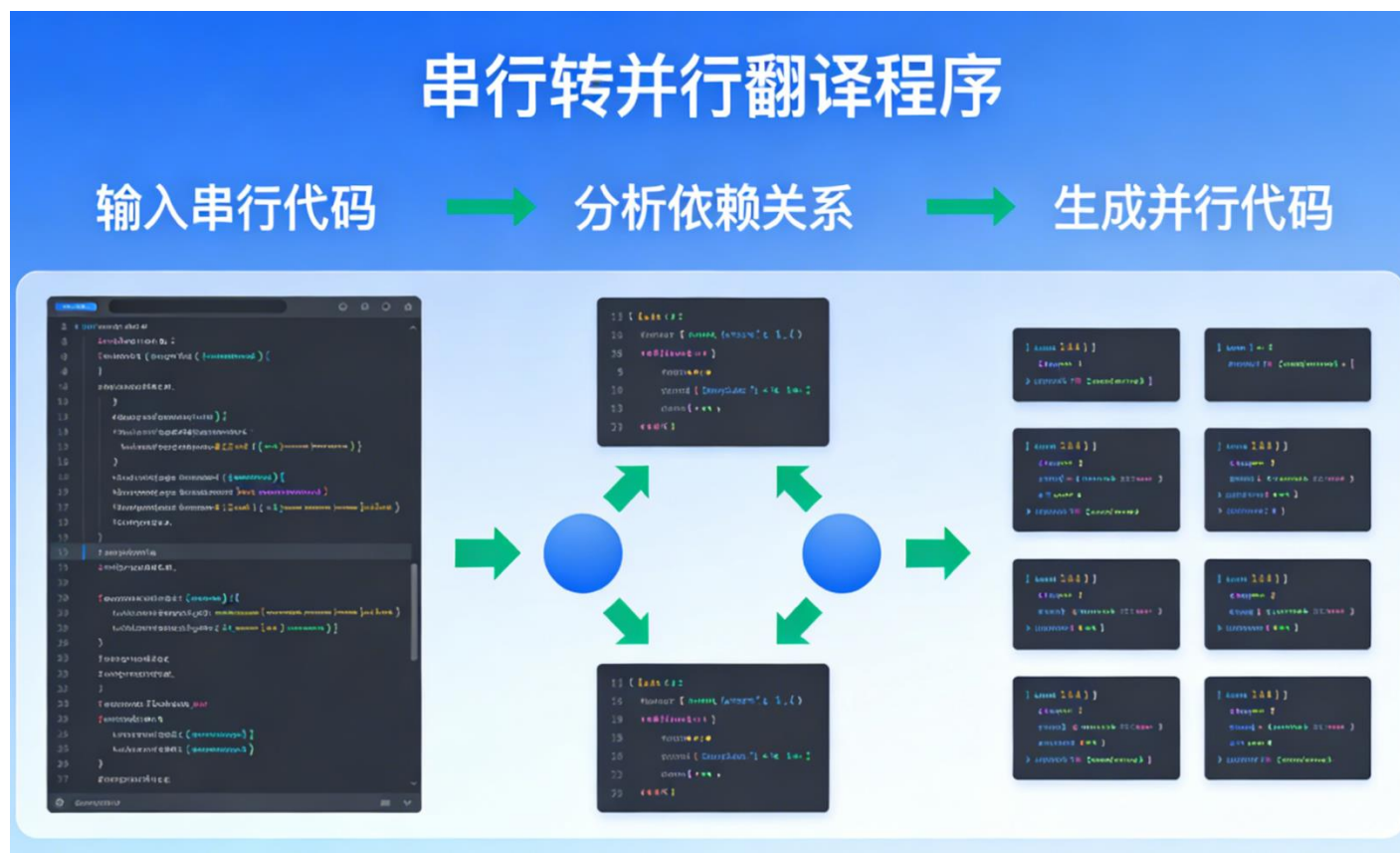
为什么需要编写并行程序

- 可以在多核系统上运行一个程序的多个实例，但这往往没有什么帮助。
- 能够运行我们最喜欢的游戏的多个实例，并不是我们真正想要的。
- 我们希望程序能够运行得更快且图形更逼真。



解决方法

- 重写串行程序， 使其成为并程序序。
- 编写翻译程序， 将串行程序自动转换为并程序序。
 - ▶ 很难做到。
 - ▶ 成效有限。



更多问题

- 虽然可以编写程序来识别串行程序中的常见结构，并自动将这些结构转化为有效的并行结构，但并行结构的序列可能非常低效。
- 串行程序的有效并行实现不是通过寻找其每个步骤的有效并行化而获得的。
- 相反，最好的并行化是通过设计一种全新的算法来实现。

示例

- 要计算n个值并将其相加。
- 串行求解：

```
sum = 0;
for (i = 0; i < n; i++) {
    x = Compute_next_value(. . .);
    sum += x;
}
```


示例

- 有 p 个核心，且 $p \ll n$ 。
- 每个核心可以形成大约 n/p 个值的部分和。

```
my_sum = 0;  
my_first_i = . . . ;  
my_last_i = . . . ;  
for (my_i = my_first_i; my_i < my_last_i; my_i++) {  
    my_x = Compute_next_value(. . .);  
    my_sum += my_x;  
}
```

每个核心都在使用自己的私有变量，独立于其他核心执行这段代码。

示例

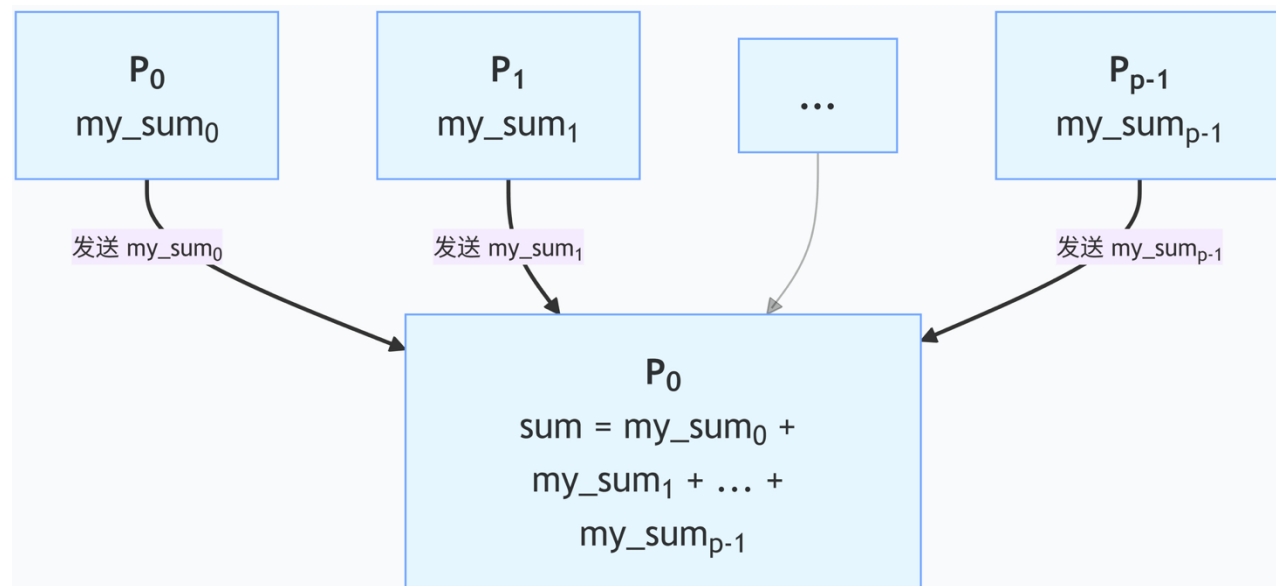
- 在每个核心完成这段代码的执行后，变量`my_sum` 将存储其调用 `Compute_next_value`所计算的值的总和。
- 例如，如果有8个核心， $n=24$ ，并且24次对`Compute_next_value`的调用返回值为：

1,4,3, 9,2,8, 5,1,1, 6,2,7, 2,5,0, 4,1,8, 6,5,1, 2,3,9

示例

- 当各核心完成对my_sum值的计算后，通过将其结果发送到指定的主核心，形成一个全局性的和。

```
if (我是主核心) {  
    sum = my_sum;  
    for 每个核心除了我自己 {  
        接收到核心传来的值;  
        sum += value;  
    }  
} else {  
    将my_sum传给主核心;  
}
```



示例

核心	0	1	2	3	4	5	6	7
my_sum	8	19	7	15	7	13	12	14

全局和

$$8 + 19 + 7 + 15 + 7 + 13 + 12 + 14 = 95$$

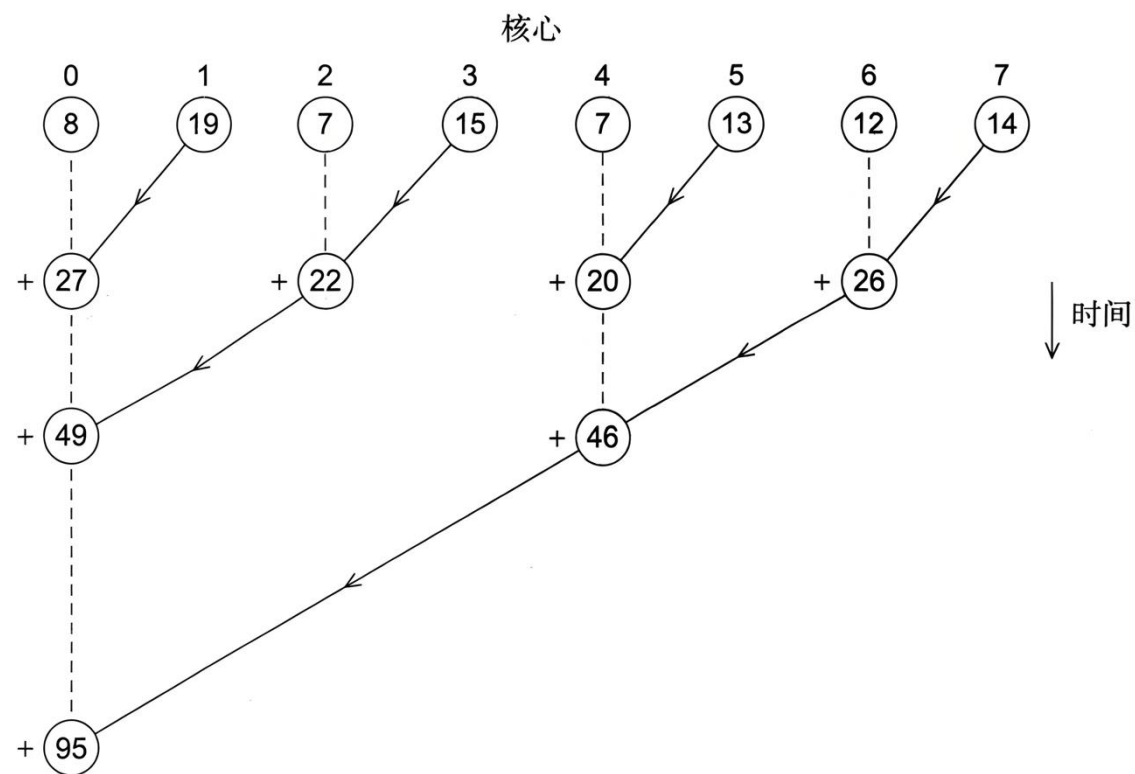
核心	0	1	2	3	4	5	6	7
my_sum	95	19	7	15	7	13	12	14

**但是等等！
有一个更好的方法
计算全局和。**



更好的并行算法

- 不让主核心做所有的求和工作。
- 核心配对求和。



多个核心形成一个全局性的和

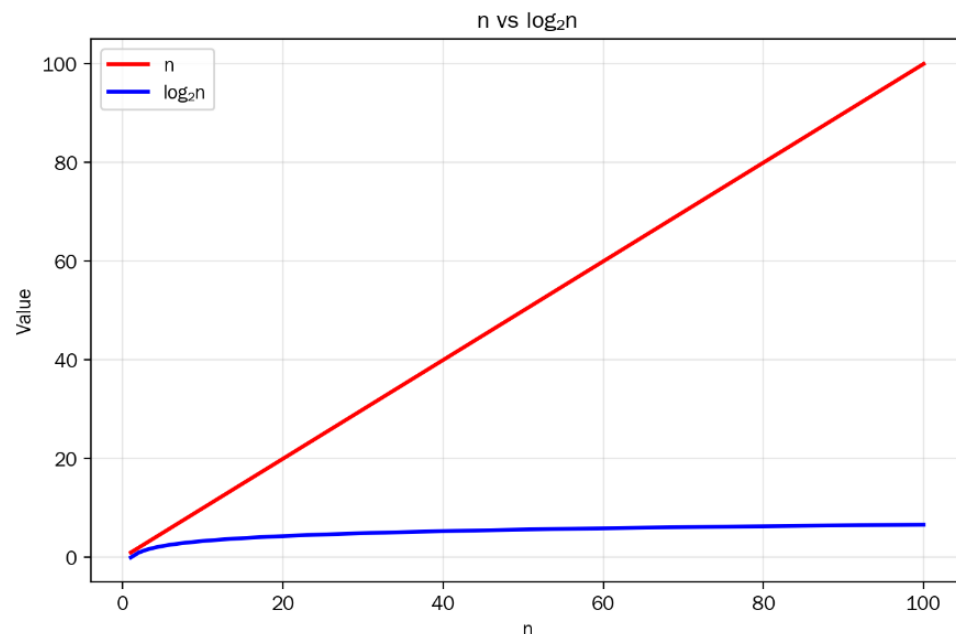
分析

- 程序时间 = 主核心完成的时间。
- 在第一个示例中，主核心执行了7次接收和7次加法。
- 在第二个示例中，主核心执行了3次接收和3次加法。
- 改进幅度超过2倍！

分析

- 当核心数量较多时，差异更为显著。
- 如果有1000个核心：
 - ▶ 第一个例子需要主核心执行999次接收和999次加法操作。
 - ▶ 第二个例子只需要10次接收和10次加法。
- 这几乎是一个100倍的改进！

n vs $\log_2 n$



如何编写并行程序？

■ 任务并行

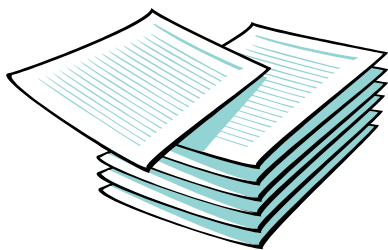
- ▶ 将解决问题时执行的各种任务划分给各个核心。

■ 数据并行

- ▶ 将用于解决问题的数据划分给各核心，
- ▶ 每个核心对其数据部分进行或多或少的类似操作。

P教授

15个问题
300份试卷



P教授的评分助手



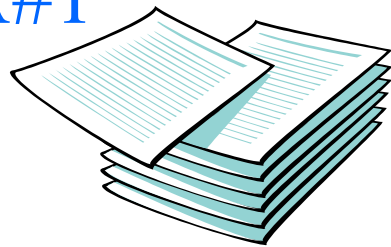
TA#1

TA#2

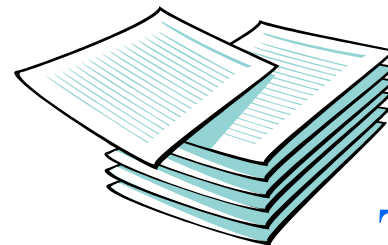
TA#3

分工 - 数据并行

TA#1

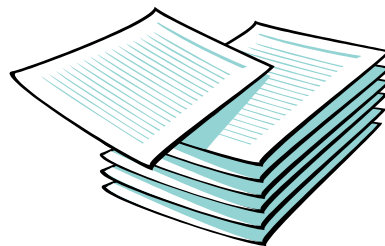


100份试卷



TA#3

100份试卷



TA#2

100份试卷

分工 - 任务并行

TA#1



问题1-5



TA#3

问题11-15



TA#2

问题6-10

分工 - 数据并行

- 每个核心对其分配的元素进行大致相同的操作：
 - ▶ 通过调用Compute_next_value 来计算所需的值,
 - ▶ 将它们累加在一起。

```
my_sum = 0;
my_first_i = . . . ;
my_last_i = . . . ;
for (my_i = my_first_i; my_i < my_last_i; my_i++) {
    my_x = Compute_next_value(. . .);
    my_sum += my_x;
}
```

分工 - 任务并行

■ 主核心：

- 接收并累加该核心的部分和。

■ 其他核心：

- 将部分和交给主核心。

```
if (我是主核心) {  
    sum = my_sum;  
    for 每个核心除了我自己 {  
        接收到核心传来的值;  
        sum += value;  
    }  
} else {  
    将my_sum传给主核心;  
}
```

协调

- 各个核心通常需要协调它们的工作。
- **通信** - 一个或多个核心将其当前的部分和发送到另一个核心。
- **负载均衡** - 在各个核心之间均匀分配工作，以确保没有一个核心负载过重。
- **同步** - 由于每个核心以自己的速度工作，因此要确保核心之间不会差距过大。

```
if (我是主核心)
    for (my_i = 0; my_i < n; my_i++)
        scanf("%lf", &x[my_i]);
for (my_i = my_first_i; my_i < my_last_i; my_i++)
    my_sum += x[my_i];
```

我们将做什么

- 学习编写**显式并行**的程序。
- 学习使用C语言。
- 学习使用C语言的四种不同扩展：
 - ▶ 消息传递接口 (MPI)
 - ▶ POSIX 线程 (Pthreads)
 - ▶ OpenMP
 - ▶ CUDA

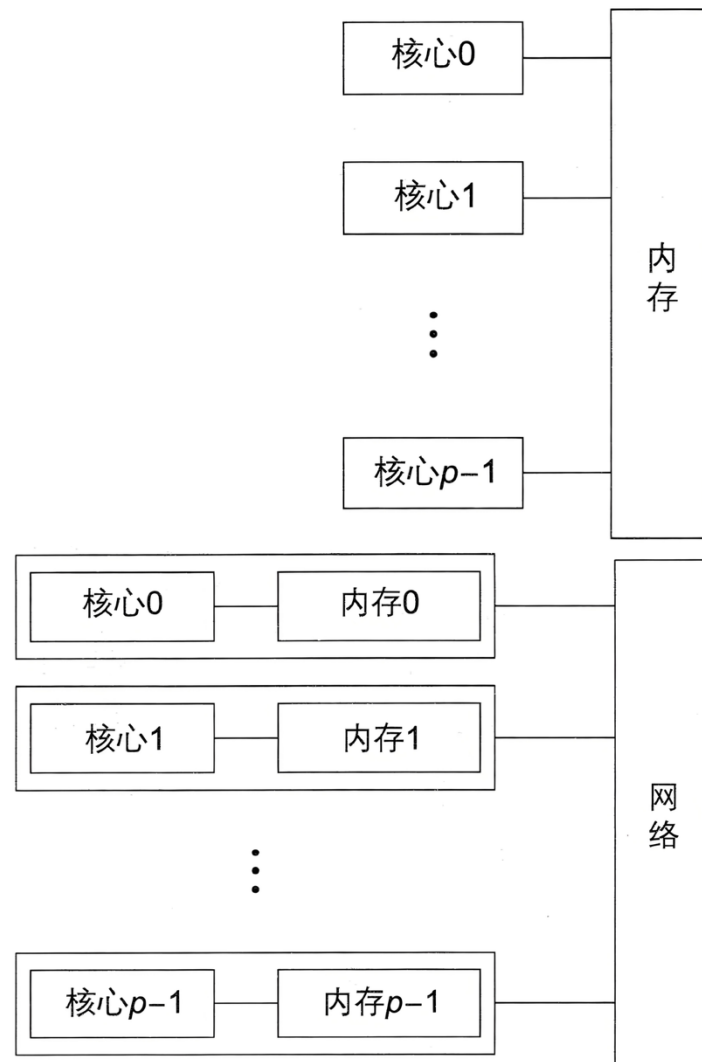
并行系统的类型

■ 共享内存系统

- ▶ 各核心可以共享对计算机内存的访问。
- ▶ 每个核心可以读写每个内存位置。
- ▶ 可以通过让核心检查和更新共享内存位置的方式来协调它们。

■ 分布式内存系统

- ▶ 每个核心都有自己的私有内存。
- ▶ 核心可以通过在网络上发送消息等方式进行显式通信。



并行系统的类型

- 根据独立指令流的数量和独立数据流的数量来划分。
- MIMD - 多指令流多数据流系统。
 - ▶ 每个核心可以管理自己的指令流和数据流。
- SIMD - 单指令流多数据流系统。
 - ▶ 共享一个控制单元。
 - ▶ 都对自己的数据执行相同的指令。

时间	第一个核心	第二个核心
1	加法	空闲
2	空闲	乘法

```
double x[n], y[n], z[n];  
...  
for (int i = 0; i < n; i++)  
    z[i] = x[i] + y[i];
```

```
i = Compute_my_subscript();  
z[i] = x[i] + y[i];
```

四种API

- MPI - 用于分布式内存MIMD系统编程的API。
- Pthreads - 用于共享内存MIMD系统编程的API。
- OpenMP - 用于共享内存MIMD和共享内存SIMD系统编程的API。
- CUDA - 用于NVIDIA GPU编程的API, 它包含四种分类：共享内存、分布式内存、SIMD和MIMD。

并发、并行与分布式计算区别对比

并发计算

定义：多个任务在同一时间段内交替执行

特点：共享单一处理器资源



示例：多任务操作系统

并行计算

定义：多个任务在同一时刻同时执行

特点：使用多个处理器并行处理



示例：超级计算机

分布式计算

定义：多个独立计算机通过网络协同完成任务

特点：节点间通过消息传递通信



示例：云计算系统

小结

- 物理定律引领我们走到了多核技术的大门前。
- 串行程序通常无法从多核中受益。
- 从串行程序代码自动生成并行程序，并非从多核计算机中获取高性能的最高效方法。
- 学习编写并行程序，需要掌握如何协调各个核心。
- 并行程序通常非常复杂，因此在并行程序中使用良好的程序开发技术就显得更加重要。